把手写策略压成调用栈，而不是塞进一个更长的 prompt

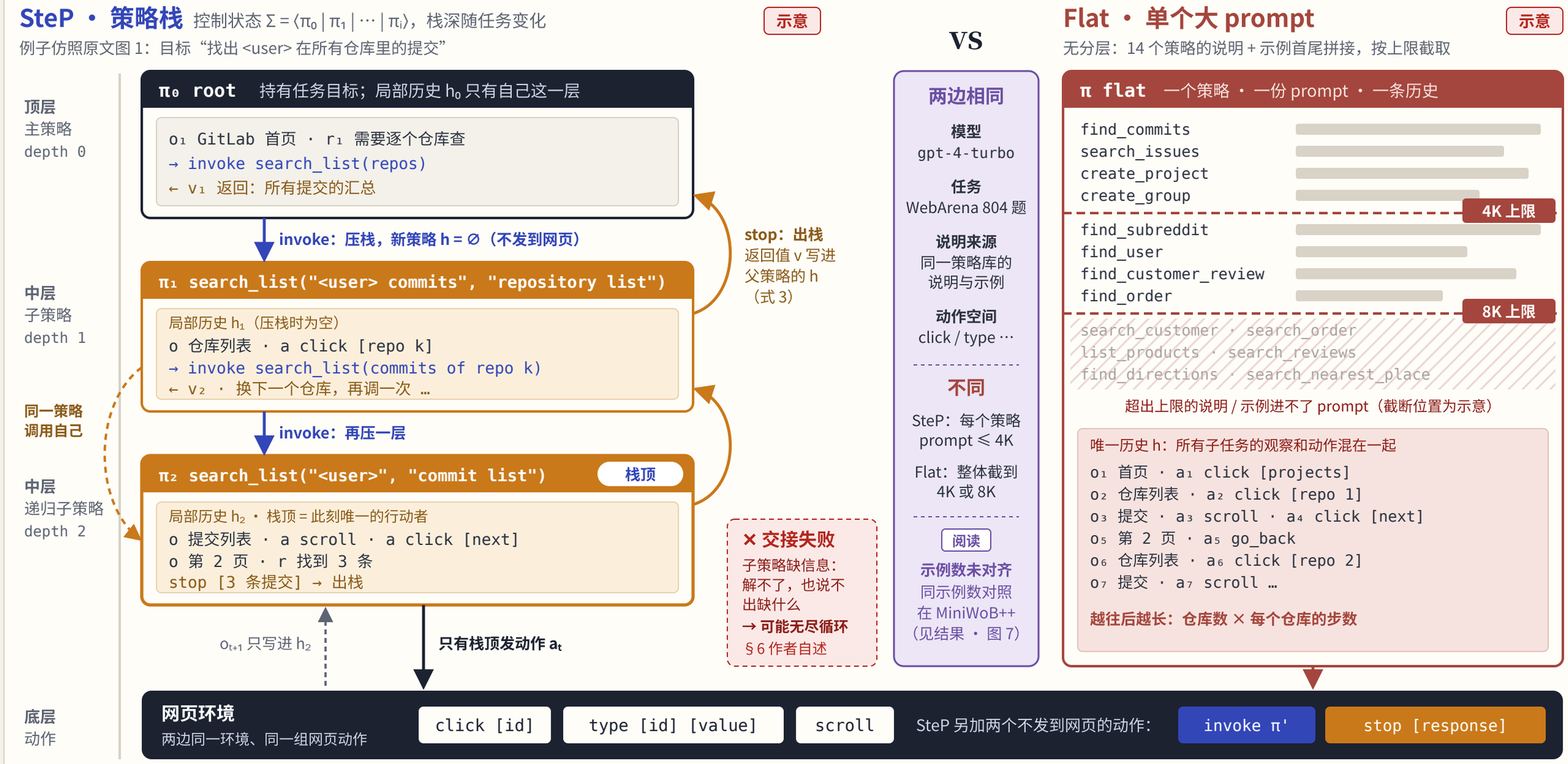
WebArena 全量：SteP 0.33；同一批说明拼进单个 prompt，Flat-4K 0.20 → Flat-8K 0.23

问题 网页任务组合爆炸。把所有任务的说明和示例写进一个 prompt，无关行为相互“泄漏”；长任务的历史越积越长，模型更难抓住要点（§1）。

做法 手写策略库（WebArena 14 个），策略间可互相调用、包括自己；调用链存成栈，只有栈顶行动，被调策略从空历史开始，结束时把返回值交回父策略（§4.1）。

机制 · 分层的策略栈 vs 平铺的大 prompt

§4.1 · 图 1 · §5.1 Baselines · 附录 A.2



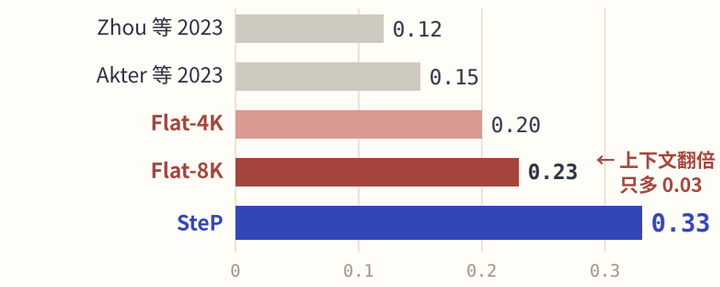
阅读　分层同时带来两件事：每次调用只装一个策略的说明（prompt 短），每层只记自己子任务的历史（历史短）。论文没有把这两件事、以及“动态递归 vs 固定层级”分开测；结果能确认的是“同一批说明拆开用，比拼进一个更长的 prompt 好”。

结果 · 同说明来源与同示例数，两组对照分开看

表 1 · 图 3 · 图 7 · §5.1.3–5.1.5

WebArena 成功率

表 1，804 题全量均值 · gpt-4-turbo

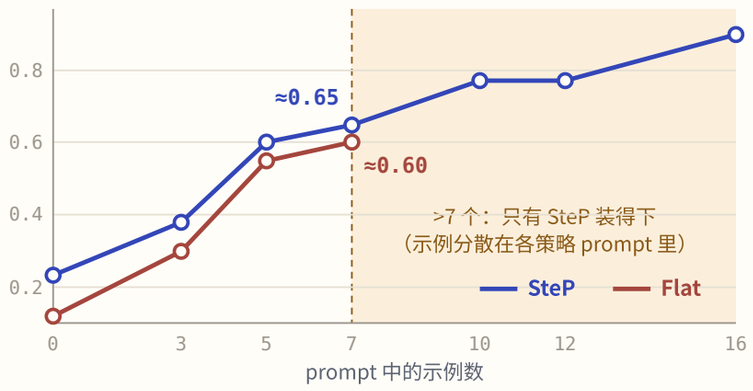


各站点均值	Flat-4K	Flat-8K	SteP
Shopping（48 intent）	0.28	0.30	**0.37**
CMS（41）	0.17	0.18	**0.24**
Reddit（21）	0.17	0.30	**0.59**
GitLab（41）	0.20	0.23	**0.32**
Maps（29）	0.20	0.20	**0.30**

原文　摘要写作 14.9% → 33.5%，对应表 1 的 Akter 等 0.15 与 SteP 0.33。每个 intent 只有 3–5 题，单次运行，无方差。

同示例数：拆开仍更好

图 7 · MiniWoB++ 子集 · text-davinci-003



同为 0 个示例：Flat ≈0.12，SteP ≈0.23；同为 7 个：≈0.60 对 ≈0.65。

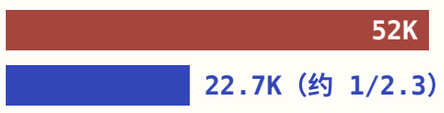
原文　示例数 ⩽7 时的提升来自“把任务说明拆成细粒度策略说明”；>7 时来自每个策略 prompt 能放更多专用示例（§5.1.5）。

示意　按原图重绘，空心点为读图约数，原文未给数值。

成本：token 少，步数和调用多

图 3(b)、表 1、§5.1.4 · 仅统计成功轨迹

平均输入 token / 成功轨迹



平均动作数（表 1 均值）

Flat-8K 7.25 · SteP **10.0**

原文　控制权来回交接，模型调用次数更多（未给次数）；因每次 prompt 短，总 token 更少。

阅读　直方图只覆盖成功轨迹，失败轨迹（含交接循环）的花费没有统计，“总成本更低”的结论只在这一范围内成立。

加长 prompt 反而退化的 intent

表 1 抽样 intent · Flat-4K → Flat-8K

intent	4K	8K
Shopping · Config of the {product} I bought	0.20	**0.00**
Shopping · Summarize main criticisms of product	0.60	**0.40**
Reddit · Like submissions {user} in {subreddit}	0.50	**0.33**
Reddit · Create new {forum} with {description}	0.60	**0.40**

原文　作者归因于 prompt 变大、难以集中注意力（§5.1.3），未做单独诊断。

SteP 也不是每个 intent 都赢

表 1 抽样 intent · Flat-8K → SteP

intent	Flat-8K	SteP
Reddit · Reply to {post} with {content}	0.60	**0.00**
Shopping · Summarize main criticisms of product	0.40	**0.00**
CMS · How many reviews received on {time}	0.60	**0.20**
Reddit · Post { notice } in {subreddit}	0.00	**0.60**

阅读　均值之下逐 intent 起落很大；每个 intent 仅 3–5 题，单格粒度粗。

限制与适用范围

§6 · §5.1 · 我们的阅读

1. **策略靠手写。**每个新领域都要人找出常见子任务、为每个写 prompt（§6）。14 个策略覆盖 ⩾50/170 个 intent；阅读 提升与“写进去的领域知识”绑在一起，不能直接外推到策略库没覆盖的任务。
2. **交接可能卡死。**子策略缺信息时既解不了、也告诉不了父策略缺什么，可能无尽循环（§6）。何时 return 由子策略自己决定，没有独立的完成校验。
3. **没拆开的因素。**没有“固定层级 vs 动态递归”的消融，也没有分别测“prompt 变短”和“历史局部化”。阅读 能确认的是分解优于单个大 prompt，不是递归本身更优。
4. **证据粒度。**WebArena 结果为单次运行、无方差；同示例数对照只在 MiniWoB++ 子集、旧模型 text-davinci-003 上做，数值只能读图。

来源：Sodhi, Branavan, Artzi, McDonald, *SteP: Stacked LLM Policies for Web Actions*, arXiv 2310.03720v4（COLM 2024）：§1、§4.1–4.2、§5.1.1–5.1.5、§6、表 1、图 1/3/7、附录 A.1–A.2。初筛：run gui-pc11-20261022-01，selection2 第 4 行，L2 见 batches/l60.jsonl。原文 = 作者报告的结果　阅读 = 我们的判断　示意 = 自绘：策略栈各帧的历史内容、Flat 截断位置、图标均为示意，不是原文图 1 的复刻；图 7 为按原图读点重绘。没有运行论文代码，也没有复现实验。